LAB REPORT: MULTITHREADING IN C

Course: Operating Systems

Student Name: Aryan Nautiyal **Roll Number:** 251210015

Aim

To explore multithreading in C using the POSIX threads (pthreads) library through three exercises: spawning N threads that identify themselves, coordinating two threads to print even and odd numbers alternately, and generating prime numbers within a range by dividing the workload across multiple threads while comparing its speed against a sequential approach.

Theory

A thread is the smallest unit of execution within a process. Multiple threads inside the same process share the address space, file descriptors, and heap, but each has its own stack and registers. This makes threads lighter than processes and useful for exploiting multi-core CPUs.

The POSIX threads API in C provides `pthread_create` to spawn threads, `pthread_join` to wait for their completion, and synchronization primitives like `pthread_mutex_t` and `pthread_cond_t` to coordinate access to shared data. Without synchronization, race conditions can produce incorrect output when threads read and write shared variables concurrently.

For CPU-bound workloads such as prime number generation, dividing the range into disjoint chunks and assigning each to a separate thread allows parallel execution on multiple cores. The expected speedup is bounded by the number of physical cores available and the overhead of thread creation and joining.

Program 1: N Threads Printing Their ID

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void *greet(void *arg) {
    long tid = (long)arg;
    printf("Hello from Thread %ld\n", tid);
    return NULL;
}

int main() {
    int N;
    printf("Enter number of threads: ");
    scanf("%d", &N);

    pthread_t *threads = malloc(N * sizeof(pthread_t));
    for (long i = 0; i < N; i++)
        pthread_create(&threads[i], NULL, greet, (void *)(i + 1));
    for (int i = 0; i < N; i++)
        pthread_join(threads[i], NULL);

    free(threads);
    return 0;
}
```

Terminal Output

```
Enter number of threads: 5
Hello from Thread 1
Hello from Thread 3
Hello from Thread 2
Hello from Thread 5
Hello from Thread 4
```

Program 2: Even and Odd Numbers Using Threads

```c
#include <stdio.h>
#include <pthread.h>

#define LIMIT 10

int counter = 1;
pthread_mutex_t lock;
pthread_cond_t cond;

void *print_odd(void *arg) {
    while (1) {
        pthread_mutex_lock(&lock);
        while (counter % 2 == 0 && counter <= LIMIT)
            pthread_cond_wait(&cond, &lock);
        if (counter > LIMIT) {
            pthread_cond_broadcast(&cond);
            pthread_mutex_unlock(&lock);
            break;
```

```
        }
        printf("Odd  Thread: %d\n", counter++);
        pthread_cond_broadcast(&cond);
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

void *print_even(void *arg) {
    while (1) {
        pthread_mutex_lock(&lock);
        while (counter % 2 != 0 && counter <= LIMIT)
            pthread_cond_wait(&cond, &lock);
        if (counter > LIMIT) {
            pthread_cond_broadcast(&cond);
            pthread_mutex_unlock(&lock);
            break;
        }
        printf("Even Thread: %d\n", counter++);
        pthread_cond_broadcast(&cond);
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_mutex_init(&lock, NULL);
    pthread_cond_init(&cond, NULL);

    pthread_create(&t1, NULL, print_odd, NULL);
    pthread_create(&t2, NULL, print_even, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock);
    pthread_cond_destroy(&cond);
    return 0;
}
```

Terminal Output

```
Odd  Thread: 1
Even Thread: 2
Odd  Thread: 3
Even Thread: 4
Odd  Thread: 5
Even Thread: 6
Odd  Thread: 7
Even Thread: 8
Odd  Thread: 9
Even Thread: 10
```

Program 3: Prime Generation — Sequential vs Multithreaded

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <math.h>
#include <time.h>

int N;
int *prime_count_per_thread;

int is_prime(int n) {
    if (n < 2) return 0;
    if (n == 2) return 1;
    if (n % 2 == 0) return 0;
    for (int i = 3; i <= (int)sqrt(n); i += 2)
        if (n % i == 0) return 0;
    return 1;
}

typedef struct { int start, end, tid; } Range;

void *find_primes(void *arg) {
    Range *r = (Range *)arg;
    int count = 0;
```

```c
    for (int i = r->start; i <= r->end; i++)
        if (is_prime(i)) count++;
    prime_count_per_thread[r->tid] = count;
    return NULL;
}

int sequential_count(int limit) {
    int count = 0;
    for (int i = 2; i <= limit; i++)
        if (is_prime(i)) count++;
    return count;
}

int main() {
    printf("Enter N: ");
    scanf("%d", &N);

    struct timespec s1, e1;
    clock_gettime(CLOCK_MONOTONIC, &s1);
    int seq_total = sequential_count(N);
    clock_gettime(CLOCK_MONOTONIC, &e1);
    double seq_time = (e1.tv_sec - s1.tv_sec) + (e1.tv_nsec - s1.tv_nsec) / 1e9;

    int num_threads = (N + 99) / 100;
    pthread_t *threads = malloc(num_threads * sizeof(pthread_t));
    Range *ranges = malloc(num_threads * sizeof(Range));
    prime_count_per_thread = calloc(num_threads, sizeof(int));

    struct timespec s2, e2;
    clock_gettime(CLOCK_MONOTONIC, &s2);
    for (int i = 0; i < num_threads; i++) {
        ranges[i].start = i * 100 + 1;
        ranges[i].end = (i + 1) * 100;
        if (ranges[i].end > N) ranges[i].end = N;
        if (ranges[i].start < 2) ranges[i].start = 2;
        ranges[i].tid = i;
        pthread_create(&threads[i], NULL, find_primes, &ranges[i]);
    }
    int par_total = 0;
    for (int i = 0; i < num_threads; i++) {
        pthread_join(threads[i], NULL);
        par_total += prime_count_per_thread[i];
    }
    clock_gettime(CLOCK_MONOTONIC, &e2);
    double par_time = (e2.tv_sec - s2.tv_sec) + (e2.tv_nsec - s2.tv_nsec) / 1e9;

    printf("\nSequential : %d primes in %.6f s\n", seq_total, seq_time);
    printf("Threaded   : %d primes in %.6f s (%d threads)\n",
           par_total, par_time, num_threads);
    printf("Speedup    : %.2fx\n", seq_time / par_time);

    free(threads); free(ranges); free(prime_count_per_thread);
    return 0;
}
```

Terminal Output(on i3 6006u, 2 cores, 4 threads)

```
Enter N: 10000

Sequential : 1229 primes in 0.004128 s
Threaded   : 1229 primes in 0.002641 s (100 threads)
Speedup    : 1.56x

Enter N: 100000

Sequential : 9592 primes in 0.104372 s
Threaded   : 9592 primes in 0.058903 s (1000 threads)
Speedup    : 1.77x

Enter N: 500000

Sequential : 41538 primes in 1.128461 s
Threaded   : 41538 primes in 0.641285 s (5000 threads)
Speedup    : 1.76x
```